

Overview

This document explains the API documentation implementation using OpenAPI 3.0 and SpringDoc, providing interactive API documentation, automatic schema generation, and testing capabilities through Swagger UI.

Configuration

Dependencies (pom.xml)

```
<dependency> <groupId>org.springdoc</groupId>  
<artifactId>springdoc-openapi-starter-webmvc-ui</artifactId> <version>2.6.0</version>  
</dependency>
```

OpenAPIConfig.java

Located in: src/main/java/CivilControl/backEnd/config/OpenAPIConfig.java

```
@Configuration public class OpenAPIConfig { @Bean public OpenAPI customOpenAPI() {  
return new OpenAPI().info(new Info().title("CivilControl Backend API").version("1.0")  
.description("Management System API for ESEA SA").contact(new Contact().name("Maximo  
Andriola").email("maximo@example.com")).license(new License().name("Private")  
.url("https://esea-sa.com"))) .addSecurityItem(new SecurityRequirement().addList("Bearer  
Authentication")).components(new Components().addSecuritySchemes("Bearer  
Authentication", new SecurityScheme().type(SecurityScheme.Type.HTTP).scheme("bearer")  
.bearerFormat("JWT").description("Enter JWT token"))); } }
```

Controller Documentation

Basic Documentation

```
@RestController @RequestMapping("/api/v1/vehicles") @RequiredArgsConstructor  
@Tag(name = "Vehicles", description = "Vehicle management endpoints") public class  
VehicleController { private final IVehicleService vehicleService; @PostMapping  
@PreAuthorize("hasAuthority('VEHICLE_CREATE')") @Operation(summary = "Create new  
vehicle", description = "Creates a new vehicle in the system", security =  
@SecurityRequirement(name = "Bearer Authentication")) @ApiResponse(value = {
```

```

@ApiResponse( responseCode = "201", description = "Vehicle created successfully", content =
@Content( mediaType = "application/json", schema = @Schema(implementation =
VehicleResponseDTO.class) ) ), @ApiResponse( responseCode = "400", description = "Invalid
input data", content = @Content( mediaType = "application/json", schema =
@Schema(implementation = ErrorResponse.class) ) ), @ApiResponse( responseCode = "409",
description = "Vehicle already exists", content = @Content( mediaType = "application/json",
schema = @Schema(implementation = ErrorResponse.class) ) ) }) public
ResponseEntity<VehicleResponseDTO> createVehicle( @Valid @RequestBody
@io.swagger.v3.oas.annotations.parameters.RequestBody( description = "Vehicle data",
required = true ) VehicleDTO vehicleDTO) { VehicleResponseDTO response =
vehicleService.createVehicle(vehicleDTO); return
ResponseEntity.status(HttpStatus.CREATED).body(response); } @GetMapping
@PreAuthorize("hasAuthority('VEHICLE_READ')") @Operation( summary = "Get all vehicles",
description = "Retrieves paginated list of vehicles with optional filters" ) @Parameter(name =
"page", description = "Page number (0-indexed)", example = "0") @Parameter(name = "size",
description = "Page size", example = "20") @Parameter(name = "sort", description = "Sort field
and direction", example = "licensePlate,asc") public
ResponseEntity<Page<VehicleResponseDTO>> getAllVehicles( @Parameter(description =
"Filter criteria", required = false) @ModelAttribute VehicleFilterDTO filters, Pageable pageable) {
Page<VehicleResponseDTO> vehicles = vehicleService.getAllVehicles(filters, pageable); return
ResponseEntity.ok(vehicles); } }

```

Annotations Reference

@Tag: Group endpoints by category

@Tag(name = "Vehicles", description = "Vehicle management endpoints")

@Operation: Describe endpoint

@Operation(summary = "Short description", description = "Detailed description", security =
@SecurityRequirement(name = "Bearer Authentication"))

@ApiResponse / **@ApiResponses:** Document responses

@ApiResponses(value = { **@ApiResponse**(responseCode = "200", description = "Success"),
@ApiResponse(responseCode = "404", description = "Not found") })

@Parameter: Document parameters

@Parameter(name = "id", description = "Vehicle ID", required = true, example = "1")

DTO Schema Documentation

Schema Annotations

@Schema(description = "Vehicle data transfer object") public record VehicleDTO(**@Schema**(

```
description = "Vehicle license plate", example = "ABC123", required = true, pattern =
"^[A-Z]{3}\\d{3}$" ) @NotBlank(message = "{vehicle.licensePlate.required}") String licensePlate,
@Schema( description = "Vehicle brand", example = "Toyota", required = true )
@NotNull(message = "{vehicle.brand.required}") String brand, @Schema( description = "Vehicle
model", example = "Hilux", required = true ) @NotNull(message = "{vehicle.model.required}")
String model, @Schema( description = "Manufacturing year", example = "2023", minimum =
"2000", maximum = "2100" ) @NotNull(message = "{vehicle.year.required}") Integer year,
@Schema( description = "Type of vehicle", example = "TRUCK", allowableValues = {"CAR",
"TRUCK", "VAN", "MOTORCYCLE"} ) VehicleType vehicleType ) {}
```

Accessing Documentation

Swagger UI

URL: <http://localhost:8081/swagger-ui.html>

Features:

- Interactive API testing
- Request/response examples
- Schema exploration
- Authentication support

OpenAPI JSON

URL: <http://localhost:8081/v3/api-docs>

Format: OpenAPI 3.0 specification in JSON format

Testing with Swagger UI

1. Authenticate

Step 1: Click “Authorize” button

Step 2: Enter JWT token:

Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Step 3: Click “Authorize”

2. Test Endpoint

Step 1: Select endpoint

Step 2: Click “Try it out”

Step 3: Fill in parameters/body

Step 4: Click “Execute”

Step 5: View response

Best Practices

1. Document All Public Endpoints

```
// Good @Operation(summary = "Get vehicle by ID") @GetMapping("/{id}") public
ResponseEntity<VehicleResponseDTO> getVehicle(@PathVariable Long id) { // ... } // Avoid -
No documentation @GetMapping("/{id}") public ResponseEntity<VehicleResponseDTO>
getVehicle(@PathVariable Long id) { // ... }
```

2. Provide Examples

```
@Schema( description = "User email", example = "user@example.com" ) String email;
```

3. Document Error Responses

```
@ApiResponses(value = { @ApiResponse(responseCode = "200", description = "Success"),
@ApiResponse(responseCode = "400", description = "Invalid input"),
@ApiResponse(responseCode = "404", description = "Not found"),
@ApiResponse(responseCode = "500", description = "Server error") })
```

4. Group Related Endpoints

```
@Tag(name = "Vehicles", description = "Vehicle operations") @Tag(name = "Maintenance",
description = "Vehicle maintenance")
```

Configuration Properties

application.properties

```
# Swagger UI customization springdoc.swagger-ui.path=/swagger-ui.html
springdoc.swagger-ui.enabled=true springdoc.swagger-ui.operationsSorter=method
springdoc.swagger-ui.tagsSorter=alpha springdoc.swagger-ui.tryItOutEnabled=true # API docs
springdoc.api-docs.path=/v3/api-docs springdoc.api-docs.enabled=true # Packages to scan
springdoc.packages-to-scan=CivilControl.backEnd.controller # Show actuator endpoints
springdoc.show-actuator=false
```

Benefits

- ✓ **Interactive Testing** - Test API without external tools
- ✓ **Auto-generated** - Updates with code changes
- ✓ **Client SDKs** - Generate clients from OpenAPI spec
- ✓ **API Contract** - Clear contract between frontend/backend
- ✓ **Onboarding** - Easy for new developers to understand API

Conclusion

OpenAPI/Swagger documentation provides comprehensive, interactive API documentation that stays synchronized with code changes, improving developer experience and API discoverability.

Developed by: Maximo Andriola

For: Ricardo de ESEA SA

Date: January 2026

Version: 1.0

Status: Production Ready ✓